

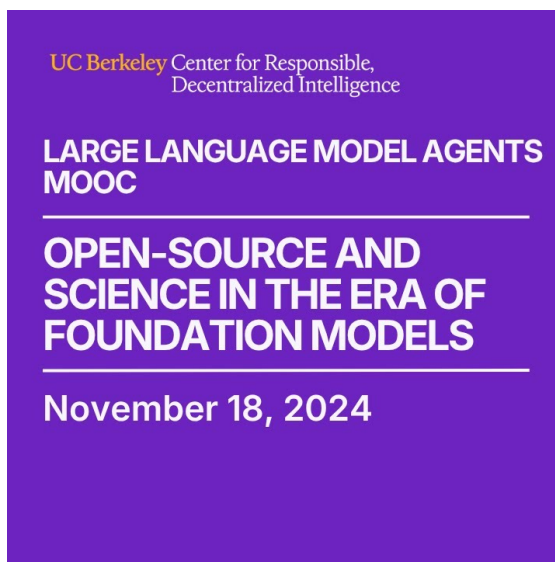
课程笔记

[LLM Agents F24] Open-Source and Science in the Era of Foundation Models

—Percy Liang

基于公开课程资料整理

2024 年 11 月 18 日



PERCY LIANG



视频作者/频道: Berkeley RDI

发布日期: 2024 年 11 月 18 日

视频时长: 约 70 分钟

视频链接: <https://www.youtube.com/live/f3KKx9LWntQ>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 讲座主线：能力上升与开放性下降的剪刀差	3
1.1 开场判断：Why Access Matters	3
1.2 历史视角：NLP 的三次“访问红利”	3
1.3 本章小结	4
2 三层访问权限：API、Open Weights、Open Source	4
2.1 三层访问模型	4
2.2 为什么要把 Open Weights 与 Open Source 区分开	5
2.3 本章小结	5
3 API 时代的 Agent：从通用函数到可执行系统	5
3.1 Universal Function 视角	5
3.2 记忆流、检索与反思循环	6
3.3 本章小结	6
4 多 Agent 编排：从结构自由度到工程纪律	7
4.1 编排维度	7
4.2 设计模式：Conversation / Tool Use / Planning / Memory	8
4.3 本章小结	8
5 Problem-Solving Agents：以 MLAGentBench 为例	8
5.1 把“会聊”变成“会做”	8
5.2 评测目标与失败分解	9
5.3 本章小结	10
6 Simulation Agents：从任务求解到社会过程建模	10
6.1 为什么要做 simulation	10
6.2 生成式社会的三层机制	10
6.3 本章小结	10
7 Open-Weight 研究窗口：解释、攻击与防御	11
7.1 可见内部带来的研究机会	11
7.2 安全侧：能力提升与风险暴露同步发生	11
7.3 本章小结	11
8 Open Source 的严格含义：从“可用”到“可复现”	11
8.1 定义争议的技术本质	11
8.2 可复现性阶梯	12
8.3 本章小结	12
9 算力与组织约束：开放科学的现实瓶颈	12
9.1 Compute 仍是第一性约束	12
9.2 去中心化协作的可行路径	13
9.3 本章小结	13

10 面向研究团队的落地清单	14
10.1 从课程到工程：如何开始搭建可复现 Agent 研发流	14
10.2 建议的评测与治理面板	15
10.3 本章小结	15
11 案例化路线图：12 周开放 Agent 研究冲刺	15
11.1 从课程理念到执行计划	15
11.2 阶段里程碑与交付物	17
11.3 建议的实验主循环（伪代码）	17
11.4 本章小结	18
12 术语与指标速查	18
12.1 核心术语对照表	18
12.2 本章小结	19
13 总结与延伸	19
13.1 整场讲座总结表	19
13.2 延伸阅读	19
13.3 收束观点	20

1 讲座主线：能力上升与开放性下降的剪刀差

1.1 开场判断：Why Access Matters

Percy Liang 在开场几分钟就给出整场讲座的核心命题：**Foundation Model 能力快速上升，但研究者可获得的访问权限在同步下降**。这种“能力曲线向上、开放曲线向下”的剪刀差，不只是商业策略变化，而是会直接重塑未来五年的研究议程。

Access Shapes Research

讲者用一句话概括：“*Access shapes research.*” 当你只有 API 访问时，你最容易做 Prompt Engineering；当你拥有权重和训练代码时，你才有机会做机制解释、训练改造、安全审计与架构创新。

LLM Agents

Instructors: Dawn Song & Xinyun Chen

Agentic AI Frameworks & AutoGen

Guest Speaker: Chi Wang

1

图 1: 讲座开场页：从 Agent 话题引向开放科学问题

画面证据：字幕约在 00:00:09–00:00:51，讲者明确提出 “importance of open source” 及 “access shapes research”。

1.2 历史视角：NLP 的三次“访问红利”

讲者回顾了 NLP/ML 历史中几次典型跃迁：

1. 90 年代到 00 年代：互联网文本可访问性提升，催生统计 NLP；
2. 10 年代：标注数据平台成熟，推动监督学习与基准化评测；
3. 深度学习时代：GPU 与集群算力可获得性提升，Transformer 规模化成为现实。

课程层面的隐含方法论

这段历史回顾并不是“科普插曲”，而是为后文铺垫一个方法论：**讨论模型能力时，必须同时讨论数据访问、计算访问和系统访问。**

常见误区：把开放问题简化为“是否开源权重”

开放问题不是二元开关。权重是否可下载只是一个维度。若训练数据、训练流程、评测脚本、系统日志与部署策略不可见，研究可复现性仍然不足，科学结论依旧脆弱。

1.3 本章小结

本章建立了整场讲座的总问题：我们不是在争论“开源好不好”，而是在讨论“不同访问层级会允许什么研究、阻断什么研究”，以及这种结构性差异如何影响 Agent 与 Foundation Model 的长期进展。

2 三层访问权限：API、Open Weights、Open Source

2.1 三层访问模型

讲者把访问权限分为三层，分别对应三种研究位置：

- API access：像“行为科学家”观察黑箱输入输出；
- Open weights：像“神经科学家”可探查内部表示；
- Open source：像“系统工程师”可重写整个流水线。

访问层级	典型能力	主要限制
API access	Prompt 设计、工具调用编排、黑箱评测	难以验证机制成因，难做结构级改造
Open weights	表征分析、蒸馏/微调、干预实验	仍受既有训练配方与架构约束
Open source	数据-模型-训练-评测全链路重构	对工程与算力要求高，复现成本大

表 1: 讲者提出的“访问层级”框架及其研究边界

同一模型，不同访问层级会导向不同论文类型

课程里最有价值的提醒是：研究问题并非完全由兴趣决定，也被“可做什么实验”决定。API 时代更容易出现策略/提示层论文；Open Source 时代更可能出现架构、训练目标、数据构造与系统 co-design 论文。

2.2 为什么要把 Open Weights 与 Open Source 区分开

Open Weights 不等于 Open Science

拿到权重可以做很多事情，但你仍不知道：

- 数据混合比例如何影响能力边界；
- 训练课程（curriculum）和对齐策略如何改变行为；
- 性能提升来自架构还是数据工程。

因此，Open Weights 是重要中间态，但并未抵达“可完整验证”的科学状态。

政策讨论中的概念混用风险

产业讨论里经常把“可调用 API”“可下载权重”“可复现训练”混成一个词“开放”。这会让监管与协作机制设计偏离真实技术约束，造成错误激励。

2.3 本章小结

讲者提出的三层框架能把“开放性争论”从口号转为工程问题：不同层级对应不同实验能力与不同风险暴露，不能混为一谈。

3 API 时代的 Agent：从通用函数到可执行系统

3.1 Universal Function 视角

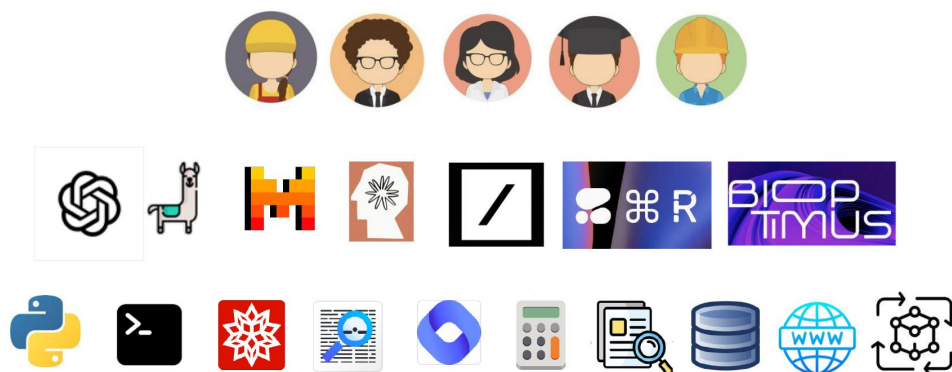
在 API-only 条件下，LLM 可以被视作“自然语言条件下的通用函数”。它在文本层面具备很强通用性，因此工程重心会转向“如何把这个函数嵌入可执行流程”。

API 不是叶子节点，而是控制器

讲者强调：在现代 Agent 系统里，LLM API 不只是固定程序里的一个子调用；它常常直接决定执行流程、工具选择与中间状态更新，实际上在扮演控制器角色。

Agentic Abstraction

Unify models, tools, human for compound AI systems



13

图 2: Agentic Abstraction: 模型、工具、人类统一抽象

画面证据：字幕约在 00:04:18-00:05:30，讲者讨论“API controls execution flow”与“memory stream / retrieval / action”。

3.2 记忆流、检索与反思循环

讲者给出的基础 Agent 架构可以拆解为四个闭环部件：观察（observation）、记忆（memory stream）、检索（retrieval）、执行（action）。进一步地，系统会加入“反思”（reflection）机制，让模型对过去轨迹做高层总结，再把总结回灌到后续决策。

为什么 Reflection 在 Agent 中如此关键

工具调用型 Agent 的失败往往不是单次推理错误，而是多步轨迹累积偏差。Reflection 的价值在于把“轨迹噪声”压缩成可复用的策略信号，使后续步骤不再盲目重复。

记忆系统最容易出现两个陷阱

- 无界累积：上下文持续膨胀导致检索质量下降；
- 错误固化：错误总结被重复引用，形成“自我强化幻觉”。

因此，记忆不是“存得越多越好”，而是“压缩质量与可验证性”更重要。

3.3 本章小结

API 时代的 Agent 关键不是“多调用几个工具”，而是构建可控的执行闭环：让 LLM 成为控制器，同时用记忆与反思机制减少长链路误差。

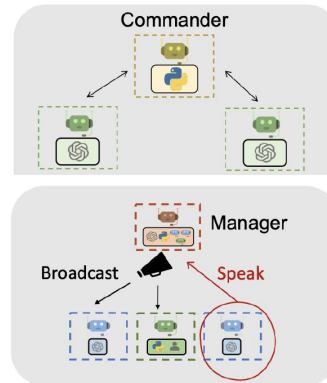
4 多 Agent 编排：从结构自由度到工程纪律

4.1 编排维度

课程中对多 Agent 编排给出了清晰的设计维度：静态/动态、集中式/去中心化、上下文共享/隔离、自动化/人工介入。这些维度决定系统性能上限与稳定性下限。

Multi-Agent Orchestration

- Static/dynamic
- NL/PL
- Context sharing/isolation
- Cooperation/competition
- Centralized/decentralized
- Intervention/automation



14

图 3: Multi-Agent Orchestration 设计维度

画面证据：字幕约在 00:06:02-00:07:40，讲者进入复杂任务求解与轨迹组织问题。

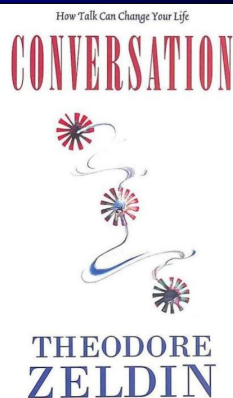
多 Agent 不是“多窗口并行”，而是“任务接口设计”

如果没有角色边界、上下文边界与验收边界，增加 Agent 数量通常只会放大冲突与重复劳动。真正有效的并行来自清晰的输入输出契约与中间产物格式。

4.2 设计模式：Conversation / Tool Use / Planning / Memory

Agentic Design Patterns

- Conversation
- Prompting & reasoning
- Tool use
- Planning
- Integrating multiple models, modalities and memories



15

图 4: Agentic Design Patterns：对话、工具、规划与记忆协同

画面证据：字幕约在 00:07:50–00:09:10，讲者连续讨论 reflection、plan、fact-check 等模式。

四类模式如何形成最小可用系统

- Conversation：承载角色协作与上下文传递；
- Tool Use：把语言决策映射到可执行动作；
- Planning：将长任务拆成可验证子目标；
- Memory：跨轮保留与压缩关键状态。

只做“模式堆叠”会导致系统复杂度失控

模式并非越多越好。每增加一种机制，都应回答两个问题：它降低了哪类失败率？它引入了哪类新故障模式？

4.3 本章小结

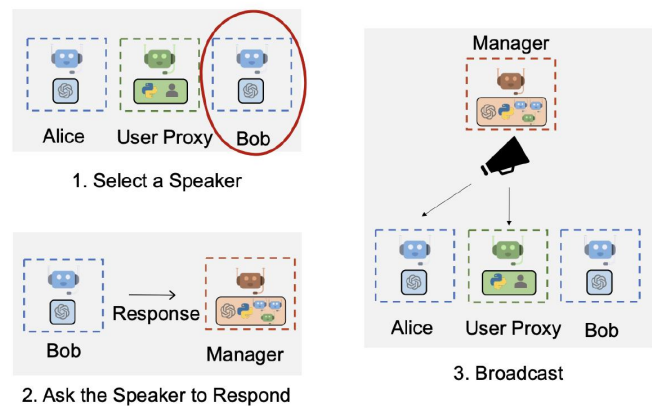
多 Agent 工程的核心不是“创意编排”，而是“约束设计”。只有接口清晰、状态可检验、回滚可执行，复杂编排才可能稳定落地。

5 Problem-Solving Agents：以 MLEAgentBench 为例

5.1 把“会聊”变成“会做”

讲者展示了 problem-solving agent 的代表方向：在真实机器学习任务中完成端到端循环，包括读任务、改代码、跑实验、分析日志、迭代策略。

Complex Task Planning and Solving with Group Chat



24

图 5: 复杂任务规划与求解：从对话到可执行 pipeline

画面证据：字幕约在 00:10:02-00:13:10，讲者讨论 benchmark 任务求解、bash 执行和自动评测。

Benchmark 价值：让“能力讨论”变成“可重复实验”

没有 benchmark，agent 改进常停留在 demo 叙事。有 benchmark 后，才能比较不同模型、不同策略、不同工具链的真实增益，并发现失败分布。

5.2 评测目标与失败分解

评测维度	可观测信号	典型失败
任务完成度	指标是否达标、提交是否有效	目标漂移、任务误解
执行可靠性	命令成功率、重试次数、超时率	环境依赖错误、工具调用失败
策略质量	迭代效率、搜索覆盖、反思有效性	盲目试错、局部最优
安全与约束	越权行为、异常副作用、审计可追踪性	未授权操作、不可回滚

表 2: Problem-solving agent 的评测不只看“最后分数”，还要看执行轨迹质量

为什么 cyber tasks 会成为关键测试场景

网络安全任务天然具备高反馈密度（成功/失败明确）与高操作风险（误操作代价高）。这类任务可以同时检验 agent 的执行能力、约束遵循能力和恢复能力。

“刷分”并不等于真实能力提升

若 benchmark 分布过窄，agent 可能学会特定套路而非通用策略。讲者强调应持续扩展任务分布与评测维度，避免出现“高分但不可迁移”的假进步。

5.3 本章小结

Problem-solving agent 的关键进展来自“可执行 + 可评测 + 可复盘”。MLAgentBench 类工作真正推动的是研究范式，而不仅是单次榜单成绩。

6 Simulation Agents：从任务求解到社会过程建模

6.1 为什么要做 simulation

讲者随后转向 simulation agents：不仅让 agent 解决任务，也让它在社会语境中持续互动，用于研究群体行为、信息传播与协作机制。

Simulation 的价值不在“像不像人”，而在“能否形成可检验假设”

当 agent 作为可编程个体参与多主体系统时，我们可以系统地改变记忆策略、沟通规则、激励函数，观察宏观行为如何变化。

6.2 生成式社会的三层机制

微观到宏观的桥梁

1. 记忆层：个体如何保留和召回经历；
2. 决策层：个体如何在上下文中行动；
3. 交互层：个体之间如何影响彼此并形成群体动态。

这三层联动才构成“可研究的社会模拟系统”。

仿真结果的外部有效性风险

仿真系统中的规则往往由研究者设定，因此结果可能更反映“规则选择”而非真实社会机制。如果没有充分的现实校验，simulation 结论应被视为假设生成，而非事实证明。

6.3 本章小结

Simulation agents 把 Agent 研究扩展到“系统行为层”。它们最有价值的产出不是故事，而是可以被实验检验和反驳的机制假设。

7 Open-Weight 研究窗口：解释、攻击与防御

7.1 可见内部带来的研究机会

进入 open-weight 层后，研究者可做的事情显著增加：表征可解释、激活分析、特征编辑、蒸馏与对齐实验都成为可能。

Open Weights 带来的是“机制研究权”，不是“万能改造权”

你可以深入模型内部，但仍被训练历史、架构形态和数据来源所约束。因此，open-weight 是强能力层，但不是终局层。

7.2 安全侧：能力提升与风险暴露同步发生

讲座中段多次提到安全相关实验：开放权重让防御研究更深入，也让攻击路径更可操作。这不是“开或不开”的单选题，而是“如何配套治理与审计机制”的系统题。

研究方向	开放权重的收益	对应风险
机制解释	可定位失败模式与脆弱特征	攻击者也可定位薄弱环节
对齐实验	可复现训练与安全干预路径	低成本复制有害变体
防御评测	可进行高强度红队与压力测试	防御细节被对抗性学习利用

表 3: Open-weight 研究中的“能力-风险同增”结构

技术社区常见误判：把风险治理外包给“模型提供方”

一旦生态进入开源/开权重形态，风险治理不能只靠单一组织。需要基准、审计工具、部署规范与责任机制共同演进。

7.3 本章小结

Open-weight 打开了机制研究和安全研究的重要窗口，但它同时放大了生态治理难度。技术进步必须与治理能力同步扩容。

8 Open Source 的严格含义：从“可用”到“可复现”

8.1 定义争议的技术本质

讲者后半段讨论了开放定义的边界问题：如果只有权重、缺少训练数据与完整流程，是否足以称为 open source？这不是语义争执，而会影响科研可复现性和责任划分。

Open Source for Models 需要新的定义框架

传统软件开源主要围绕“源代码可见可改”。模型系统中，数据、训练流程、评测协议与发布策略同样是“源”的组成部分。

8.2 可复现性阶梯

开放层级	可做的复现	仍然缺失
API 可用	行为复测、接口级评测	机制归因、训练追溯
Weights 可得	推理复现、微调复现、部分解释实验	数据与训练因果链
Full source	训练与评测端到端复现、系统级改造	主要瓶颈转为算力与工程资源

表 4: 从“可调用”到“可复现”的能力阶梯

为什么“可复现”比“可下载”更关键

科学共同体需要可验证的知识积累。只有当关键实验条件可复现、可比对、可反驳，领域才能形成稳固共识，而非靠个别组织的不可验证声明。

把“开源”当作营销标签会反噬社区信任

若术语长期被滥用，研究者会在协作预期上出现系统性误判：以为可复现，实际不可复现；以为可审计，实际不可审计。

8.3 本章小结

本章核心是“定义即基础设施”。对于模型系统，开放定义直接决定研究可复现性、协作效率与治理可操作性。

9 算力与组织约束：开放科学的现实瓶颈

9.1 Compute 仍是第一性约束

讲者指出，即便开放定义清晰、社区意愿强烈，开放科学仍会被算力与系统工程约束。这意味着未来竞争不只在模型能力，也在谁能构建可复用的训练与评测基础设施。

AutoGen is simply the gold-standard when it comes to applied enterprise agentic orchestration. It allows us to easily and rapidly explore multiple agentic configurations, conducting experiments at scale. Without this specific capability, you'll never realize the full potential of multi-agent systems. Thank you AutoGen!
Medo Eldin, Cofounder and CEO @ Terrascope AI

30

图 6: 生态采用与社区贡献：开放框架的网络效应

画面证据：字幕约在 00:26:01 与 00:31:01，讲者讨论生态规模、开放模型实践与研究迁移价值。

基础设施视角下的三类稀缺资源

- 训练算力：决定可探索的模型/数据空间；
- 工程人才：决定系统可维护性与实验吞吐；
- 评测资产：决定进展是否可验证与可比较。

9.2 去中心化协作的可行路径

开放生态的真正杠杆：共享标准与共享工具链

单点算力难以复制 frontier 训练，但社区可以通过统一评测、可移植训练脚本、可复用数据治理流程形成“协作乘数”。

没有标准化，协作会退化成碎片化重复劳动

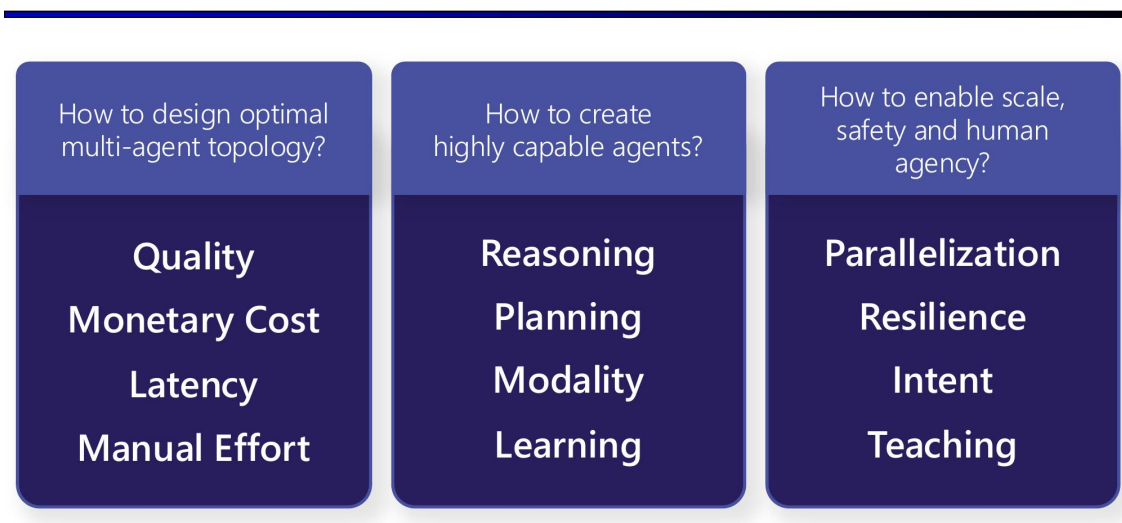
若各团队使用互不兼容的日志格式、评测协议和任务定义，结果是“每个人都在做实验，但没人能有效比较实验”。

9.3 本章小结

开放科学不等于低门槛。它需要新的共同基础设施来降低协作摩擦，否则“开放”只会停留在口号层面。

10 面向研究团队的落地清单

10.1 从课程到工程：如何开始搭建可复现 Agent 研发流



36

图 7: 研究问题收束：能力、安全、人类参与三角

画面证据：字幕约在 00:51:01–00:52:00，讲者强调系统性主题与后续研究路线。

建议的最小落地路径

1. 先定义可验证任务，再讨论 Agent 架构；
2. 先建立日志与回放，再优化策略；
3. 先做失败分解，再追求榜单分数；
4. 先统一评测协议，再扩大团队并行。

10.2 建议的评测与治理面板

面板维度	核心指标	落地建议
任务表现	pass rate, success@k, latency	固定任务版本，保留历史可回放轨迹
执行质量	工具失败率、重试率、回滚率	引入强制审计日志与失败分类标签
安全约束	越权事件、敏感操作告警	明确权限边界与人工确认节点
科学复现	结果方差、环境一致性、复现实验次数	公开配置快照与评测脚本版本

表 5: Agent 研究从“会跑”到“可积累”的监控面板

团队协作中的关键组织设计

研究负责人应把“可复现性”设为一等目标：代码提交、数据版本、模型版本、评测版本必须能形成同一条追踪链，避免“结果正确但过程不可追踪”。

不要过早追求“全自动”

在高风险、高不确定任务里，合理的人类介入不是倒退，而是提升系统稳健性的必要机制。真正目标是“可控自动化”，不是“无人值守幻想”。

10.3 本章小结

落地层面最重要的不是某个框架名称，而是：任务定义、评测协议、日志治理、权限控制四件事能否同时成立。

11 案例化路线图：12 周开放 Agent 研究冲刺

11.1 从课程理念到执行计划

为了把本讲的理念转成可操作项目，可以采用“12 周冲刺”结构：前 4 周打基础设施，中间 4 周做系统迭代，最后 4 周做稳健性和复现封装。这个节奏对应讲者的核心精神：**先把研究系统做成可验证机器，再追求局部性能峰值。**

Key Benefits of Agentic AI

- Useful Interface
 - Natural interaction with human agency
- Strong Capability
 - Operate with minimal human intervention
- Useful Architecture
 - Intuitive programming paradigm

6

图 8: Agent 系统收益目标：接口、能力、架构三位一体

画面证据：字幕约在 00:00:43-00:01:39，讲者从 “access shapes research” 延展到系统能力边界。

建议的冲刺目标不是 “上线一个酷炫 demo”

更高价值的目标是：

- 任一实验结论能被同伴在 24 小时内复现；
- 任一失败轨迹能被快速定位到具体环节；
- 任一提升都能说明来自数据、策略还是工具链。

这样做短期看慢，长期看会显著降低 “伪进步” 比例。

11.2 阶段里程碑与交付物

阶段	核心任务	强制交付物	验收标准
第 1-2 周	统一任务协议与日志格式	task schema, run schema, replay script	任意任务可回放
第 3-4 周	基线 agent 与评测跑通	baseline result, failure taxonomy	有稳定 baseline 方差
第 5-6 周	引入记忆/反思/规划模块	ablation report, trace dashboard	能解释增益来源
第 7-8 周	多 agent 编排与权限治理	orchestration spec, safety gate	越权行为可拦截可审计
第 9-10 周	鲁棒性与分布外压力测试	stress suite, adversarial suite	不同分布下失败可分解
第 11-12 周	复现封装与文档化发布	reproducibility package, model card	第三方可独立复现

表 6: 12 周冲刺的最小研究工程化路径

为什么先做 replay 再做优化

很多团队把 80% 时间花在“想新策略”，却没有可重放轨迹。没有 replay，你无法判断改进是策略有效还是随机波动；没有稳定 replay，后续所有优化都无法累积为可信知识。

11.3 建议的实验主循环（伪代码）

```
1 for task_batch in benchmark:
2     run_id = launch_agent(task_batch, config, tools, safety_policy)
3     traces = collect_traces(run_id)
4     metrics = evaluate(traces, eval_protocol)
5
6     if metrics.regression_detected:
7         rollback(config)
8         tag_failure(traces, taxonomy)
9         continue
10
11     insights = reflection_and_ablation(traces, metrics)
12     config = update_config(config, insights)
13     snapshot(run_id, config, metrics, traces)
```

Listing 1: Research Loop: 可复现 Agent 迭代主循环

最容易忽视的工程债务：不可比较的实验

如果每次迭代都更换任务集合、工具版本或评测脚本，就会出现“每次都在进步，但没有两次结果可比较”的假象。冲刺阶段必须冻结关键协议，只在受控窗口调整变量。

11.4 本章小结

这个 12 周路线图的目标是把“开放科学 + Agent 工程”的理念落地为可执行制度：可复现、可审计、可解释。只要三者缺一，团队就会在规模化阶段反复返工。

12 术语与指标速查

12.1 核心术语对照表

术语	中文理解	在本讲中的作用
API Access	黑箱接口访问	允许行为层实验，但难以做机制级归因。
Open Weights	权重开放	允许内部分析、微调和蒸馏，但仍受既有训练配方约束。
Open Source	全链路开放	覆盖数据、模型、训练和评测，支持系统级可复现。
Access Shapes Research	访问塑造研究	课程主线：可访问资源决定可提出的问题类型。
Agent Controller	执行控制器	LLM 不只是被调用模块，而是流程控制中枢。
Memory Stream	记忆流	记录历史观察与行为，为后续检索与反思提供上下文。
Retrieval	检索	从长记忆中提取当前任务最相关状态。
Reflection	反思	把历史轨迹压缩成高层策略，降低重复错误。
Planning	规划	将长任务拆解为可验证子目标。
Tool Use	工具调用	把语言决策映射为真实执行动作。
Orchestration	编排	组织多 agent 的角色边界、上下文边界和执行顺序。
Benchmark	基准评测	将“看起来更强”转化为可比较、可复验结果。
Failure Taxonomy	失败分类	区分目标误解、工具故障、策略退化等失效类型。
Replayability	可回放性	让每次失败都能被重放与诊断，是复现基础。
Audit Log	审计日志	为安全与责任追踪提供证据链。
Distribution Shift	分布偏移	评估 agent 在 OOD 场景的稳定性与泛化能力。
Safety Gate	安全闸门	对高风险动作设置权限确认与回滚机制。
Reproducibility Package	复现封装	汇总配置、脚本、环境与结果，供第三方验证。

术语	中文理解	在本讲中的作用
Open Definition	开放定义	明确“开放”边界，避免术语混用导致协作失真。
Research Throughput	研究吞吐	由算力、工具链和组织流程共同决定。

为什么要做术语速查

Agent 研究横跨模型、系统、评测与治理，不同团队常用词汇并不一致。将术语标准化可以显著降低协作误解，尤其在跨机构复现实验时效果明显。

12.2 本章小结

术语统一是低成本高收益的工程动作。它不直接提升榜单分数，但会显著提升团队的实验可比较性与知识沉淀速度。

13 总结与延伸

13.1 整场讲座总结表

主题	讲者核心观点	对 Agent 研究的含义
开放性趋势	能力提升与访问下降同时发生	研究边界被访问权限直接塑形
三层访问框架	API / Open Weights / Open Source 分层清晰	不同层级对应不同可验证实验能力
API Agent 工程	LLM 充当执行控制器	重点转向记忆、反思、轨迹治理
Benchmark 与实证	可评测体系是进步前提	从 demo 叙事转向可复现比较
开放定义与治理长期路线	术语边界影响政策与协作技术能力与基础设施共进化	“可下载”不等于“可复现” 开放科学需要标准、工具链与算力协同

表 8: Lecture 03 的核心论点压缩

13.2 延伸阅读

- Bommasani et al., *On the Opportunities and Risks of Foundation Models*, 2021.
- Liang et al., *Holistic Evaluation of Language Models (HELM)*, 2022.
- Agent evaluation and reproducibility 相关基准论文（含 MLEAgentBench 系列工作）。
- OSI 关于 AI 模型开放定义的持续讨论与草案更新。
- 多 Agent 工程框架实践：AutoGen / LangGraph / CrewAI 的设计比较。

13.3 收束观点

“*Access shapes research.*” 这句看似简单的话，实际上定义了 Agent 时代研究者的现实边界。如果希望在 Foundation Model 与 Agent 方向形成可持续科学进展，社区必须同时推进模型能力、开放访问与可复现基础设施，而不是只追逐单点 SOTA。